

Autora: Bárbara Maria Barreto Fonseca de Cerqueira César

RA: 166242

Data: Abril de 2024

1. Escopo

Projeto de um circuito combinacional, que implementa uma unidade lógica e aritmética (ULA) capaz de realizar operações com números binários de 4 bits sem sinal e com sinal, em complemento de 2.

A ULA projetada possui 13 entradas: 4 entradas para o primeiro número de 4 bits, 4 entradas para o segundo número de 4 bits, 1 entrada para o carry de entrada e 4 entradas para o código de 4 bits que selecionará a operação aritmética a ser realizada. E possui 8 saídas: 4 saídas para o resultado de 4 bits da operação entre os dois números e 4 saídas que representam cada uma “flags” distintas que são acionadas de acordo com certas condições presentes no resultado da operação.

Além disso, podemos dividir o circuito completo da ULA em 5 módulos, dos quais cada um possui seu funcionamento e propósito para o circuito. Para cada módulo, será apresentada uma descrição, o projeto do circuito e as simulações realizadas pelo software “Quartus II”.

2. O módulo “Complementa”

O bloco “complementa” é um circuito capaz de complementar cada bit de uma entrada de 4 bits, em outras palavras, ele é capaz de inverter o valor lógico assumido por aquela entrada.

2.1. Projeto do Circuito

2.1.1. Especificação Binária

Entrada:

$$i_n \in \{0, 1\}, \text{ com } n \in \{0, 1, 2, 3\}$$

$cmp \in \{0, 1\}$ - a entrada cmp funciona como um “Enable”, ou seja, ativa (quando está alta) ou desativa (quando está baixa) a função “Complementa”.

Saída:

$$o_n \in \{0, 1\}, \text{ com } n \in \{0, 1, 2, 3\}$$

Função:

$$o_n = \overline{i_n}, \text{ quando } cmp = 1$$

$$o_n = i_n, \text{ quando } cmp = 0$$

Tabela Verdade:

cmp	i_n	o_n
0	0	0
0	1	1
1	0	1
1	1	0

2.1.2. Minimizações

Queremos aplicar a função **individualmente** a cada bit, as expressões encontradas descrevem cada um desses bits. Como cada número possui 4 bits de entrada, o circuito final terá 4 vezes a quantidade de porta lógicas (uma vez para cada bit).

2.1.2.1. Soma de Produtos

	$\overline{i_n}$	i_n
$\overline{cmp}$	0	1
cmp	1	0

$$o_n = i_n \cdot \overline{cmp} + \overline{i_n} \cdot cmp \quad (2 \text{ ANDs}, 1 \text{ OR}, 2 \text{ NOTs})(5 \text{ Portas} * 4 = 20 \text{ Portas})$$

A expressão acima pode ainda ser simplificada utilizando um XOR:

$$o_n = i_n \cdot \overline{cmp} + \overline{i_n} \cdot cmp = i_n \oplus cmp$$

Logo, temos que:

$$o_n = i_n \oplus cmp \quad (1 \text{ XOR})(1 \text{ Porta} * 4 = 4 \text{ Portas})$$

2.1.2.2. Produto de Somas

	i_n	$\overline{i_n}$
cmp	0	1
$\overline{cmp}$	1	0

$$o_n = (i_n + cmp)(\overline{i_n} + \overline{cmp}) \quad (2 \text{ ORs}, 1 \text{ AND}, 2 \text{ NOTs})(5 \text{ Portas} * 4 = 20 \text{ Portas})$$

2.1.2.3. Esquemático do Circuito

O circuito será implementado utilizando a porta lógica XOR, já definida no software utilizado.

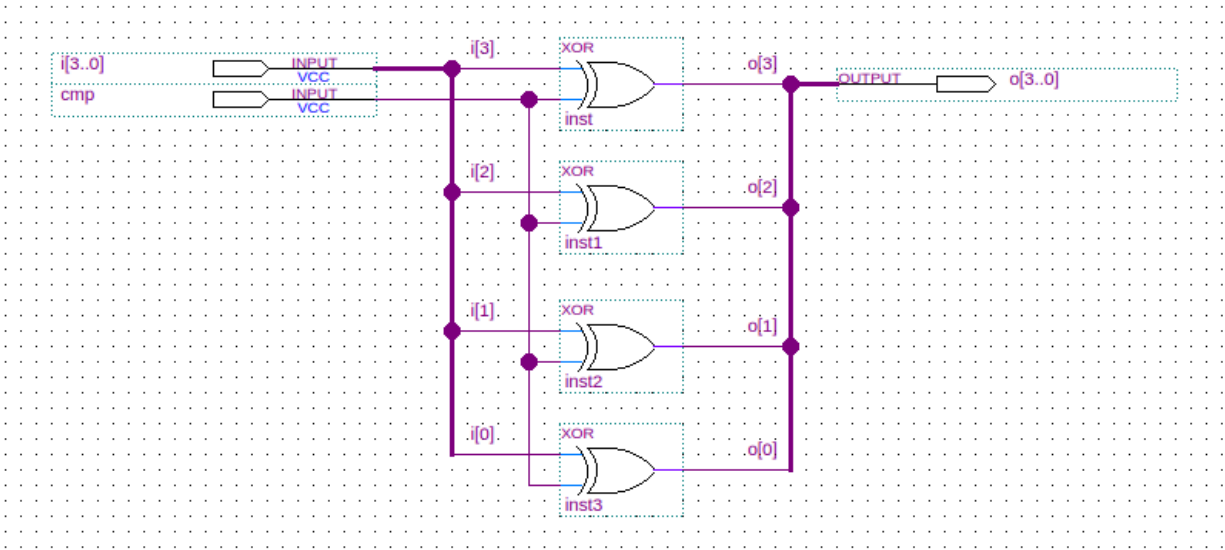


Figura 1.1: Diagrama esquemático do circuito para o módulo “Complementa”.

2.2. Simulação

Como cada número possui 4 bits, temos 16 combinações de entradas. Além disso, para cada uma das 16 combinações, há dois possíveis níveis lógicos para o enable cmp: alto (1) ou baixo (0), totalizando 32 combinações. Assim, para a simulação, as entradas foram configuradas de modo a representar todas as 32 combinações possíveis entre o enable cmp e as entradas i_n .

2.2.1. Simulação Funcional

A figura 1.2 apresenta as formas de ondas dos sinais de entrada e saída geradas pela simulação funcional. Quando cmp está baixo, as entradas são direcionadas à saída sem alteração. Quando cmp está alto, cada sinal de saída (o[0], o[1], o[2], o[3]) assume o valor contrário ao seu respectivo sinal de entrada (i[0], i[1], i[2], i[3]), ou seja, o circuito realiza o complemento individual de cada bit de entrada.

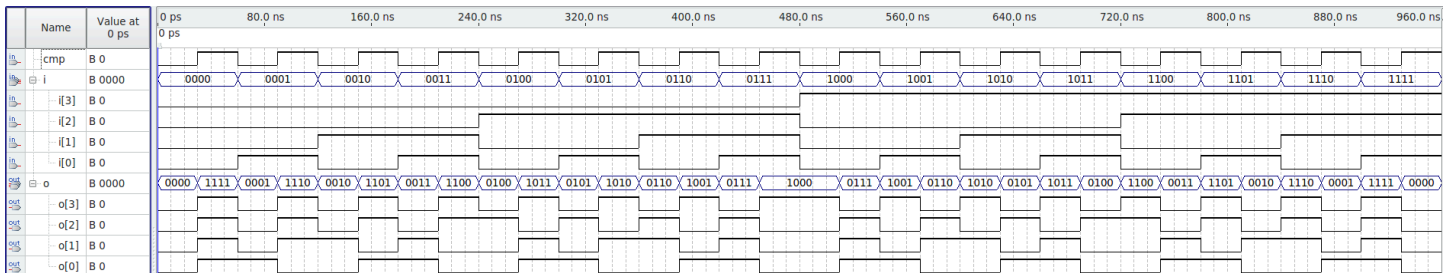


Figura 1.2: Resultado da simulação funcional para o módulo “Complementa”.

2.2.2. Simulação Temporal

A Figura 1.3 apresenta as formas de ondas dos sinais de entrada e sinal de saída geradas pela simulação temporal, que contempla os atrasos das portas. Da mesma forma que o resultado da simulação funcional, a simulação temporizada também indica a operação do circuito consoante a especificação.

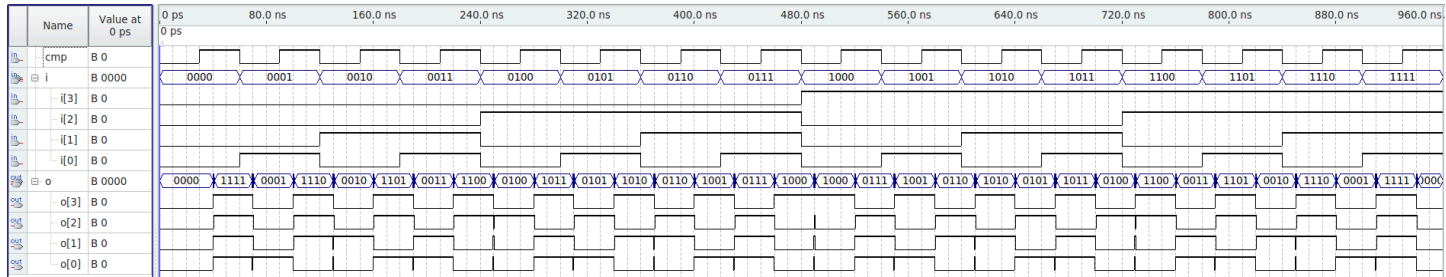


Figura 1.3: Resultado da simulação temporizada para o módulo “Complementa”.

3. O módulo “Zera”

O módulo “Zera” é um circuito combinacional que tem a função de, quando ativado, zerar todos os 4 bits de entrada.

3.1. Projeto do Circuito

3.1.1. Especificação Binária

Entrada:

$i_n \in \{0, 1\}$, com $n \in \{0, 1, 2, 3\}$

$zer \in \{0, 1\}$ - a entrada *zer* funciona como um “Enable”, ou seja, ativa (quando está alta) ou desativa (quando está baixa) a função “Zera”.

Saída:

$o_n \in \{0, 1\}$, com $n \in \{0, 1, 2, 3\}$

Função:

$o_n = 0$, quando $zer = 1$

$o_n = i_n$, quando $zer = 0$

Tabela Verdade:

zer	i_n	o_n
0	0	0
0	1	1
1	0	0
1	1	0

3.1.2. Minimizações

Queremos aplicar a função **individualmente** a cada bit, as expressões encontradas descrevem cada um desses bits. Como cada número possui 4 bits de entrada, o circuito final terá 4 vezes a quantidade de porta lógicas (uma vez para cada bit).

3.1.2.1. Soma de Produtos

	$\overline{in}$	in
$\overline{zer}$	0	1
zer	0	0

$$o_n = i_n \cdot \overline{zer} \text{ (1 AND, 1 NOT)(2 Portas * 4 = 8 Portas)}$$

3.1.2.2. Produto de Somas

	in	$\overline{in}$
zer	0	1
$\overline{zer}$	0	0

$$o_n = i_n \cdot \overline{zer} \text{ (1 AND, 1 NOT)(2 Portas * 4 = 8 Portas)}$$

3.1.2.3. Esquemático do Circuito

A função soma de produtos e a função produto de somas resultaram na mesma expressão, logo, não importa qual delas é utilizada na implementação do circuito.

Na Figura 2.1 é apresentado o diagrama esquemático do circuito projetado.

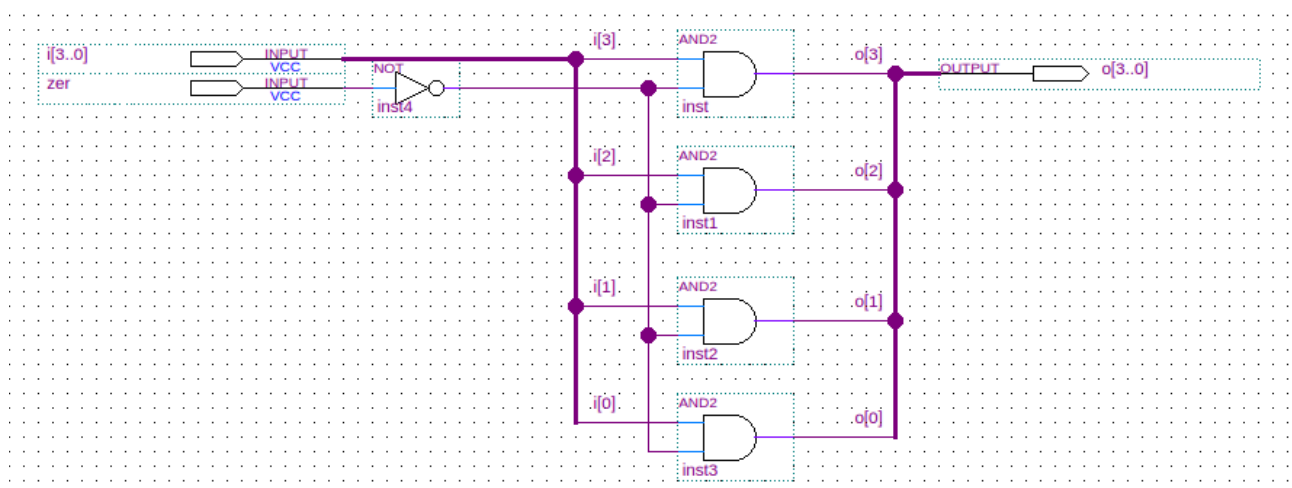


Figura 2.1: Diagrama esquemático do circuito para o módulo “Zera”.

3.2. Simulação

Como cada número possui 4 bits, temos 16 combinações de entradas. Além disso, para cada uma das 16 combinações, há dois possíveis níveis lógicos para o enable zer: alto (1) ou baixo (0), totalizando 32 combinações. Assim, para a simulação, as entradas foram configuradas de modo a representar todas as 32 combinações possíveis entre o enable zer e as entradas i_n .

2.2.1. Simulação Funcional

A figura 2.2 apresenta as formas de ondas dos sinais de entrada e saída geradas pela simulação funcional. Quando zer está baixo, as entradas são direcionadas à saída sem alteração. Quando zer está alto, todos os sinais de saída (o[0], o[1], o[2], o[3]) assumem o valor 0.

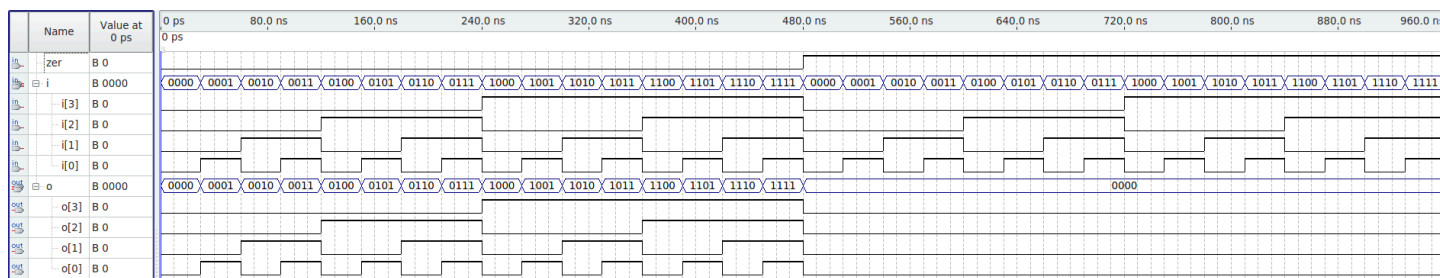


Figura 2.2: Resultado da simulação funcional para o módulo “Zera”.

3.2.2. Simulação Temporal

A Figura 2.3 apresenta as formas de ondas dos sinais de entrada e sinal de saída geradas pela simulação temporal, que contempla os atrasos das portas. Da mesma forma que o resultado da simulação funcional, a simulação temporizada também indica a operação do circuito consoante a especificação.

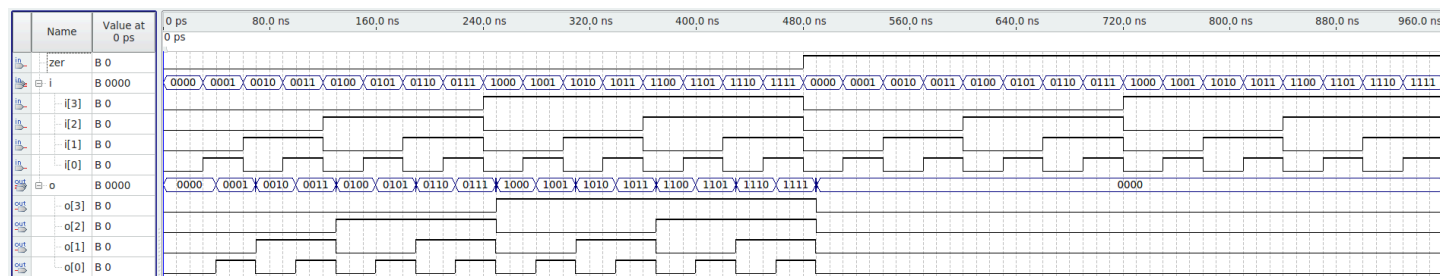


Figura 2.3: Resultado da simulação temporizada para o módulo “Zera”.

4. Módulo “Somador de 4 bits”

O módulo “Somador de 4 bits” é um circuito combinacional capaz de realizar a soma entre dois números binários de 4 bits e um carry de entrada de 1 bit. O Somador de 4 bits além de fornecer o resultado de 4 bits da operação aritmética entre os dois números, fornece também o resultado de 4 ‘flags’ distintas. Cada qual é ativada quando certas condições do resultado final são satisfeitas.

4.1. Projeto do Circuito

4.1.1. Especificação Binária

Entrada:

$$x_n, y_n \in \{0, 1\}, \text{ com } n \in \{0, 1, 2, 3\}$$

$$ci \in \{0, 1\}$$

Saída:

$s_n \in \{0, 1\}$, com $n \in \{0, 1, 2, 3\}$

$C, V, N, Z \in \{0, 1\}$

Função:

$$s_3 s_2 s_1 s_0 = x_3 x_2 x_1 x_0 + y_3 y_2 y_1 y_0$$

Além disso, temos:

c_0 : carry produzido entre $x_0 + y_0$

c_1 : carry produzido entre $x_1 + y_1$

c_2 : carry produzido entre $x_2 + y_2$

c_3 : carry produzido entre $x_3 + y_3$

Agora, vamos definir as flags:

$C = 1$, identifica se houve carry out, ou seja, quando a soma $x_3 + y_3$ (do último somador completo) produz carry de saída igual a 1, $c_3 = 1$.

$V = 1$, identifica se houve overflow ou seja, se $c_3 \oplus c_2 = 1$.

$N = 1$, identifica se o resultado da soma é um número negativo em complemento de 2, ou seja, quando $s_3 s_2 s_1 s_0 = 1xxx$.

$Z = 1$, identifica se o resultado da soma é nulo, ou seja, quando $s_3 s_2 s_1 s_0 = 0000$, ou ainda, $\overline{s_3 + s_2 + s_1 + s_0} = 1$.

Mas antes de discutirmos as “flags” do “Somador de 4 bits”, precisamos apresentar os circuitos menores que o formam e que foram encapsulados. A Figura 3.1 apresenta um esquemático do “Somador de 4 bits” e seus componentes encapsulados.

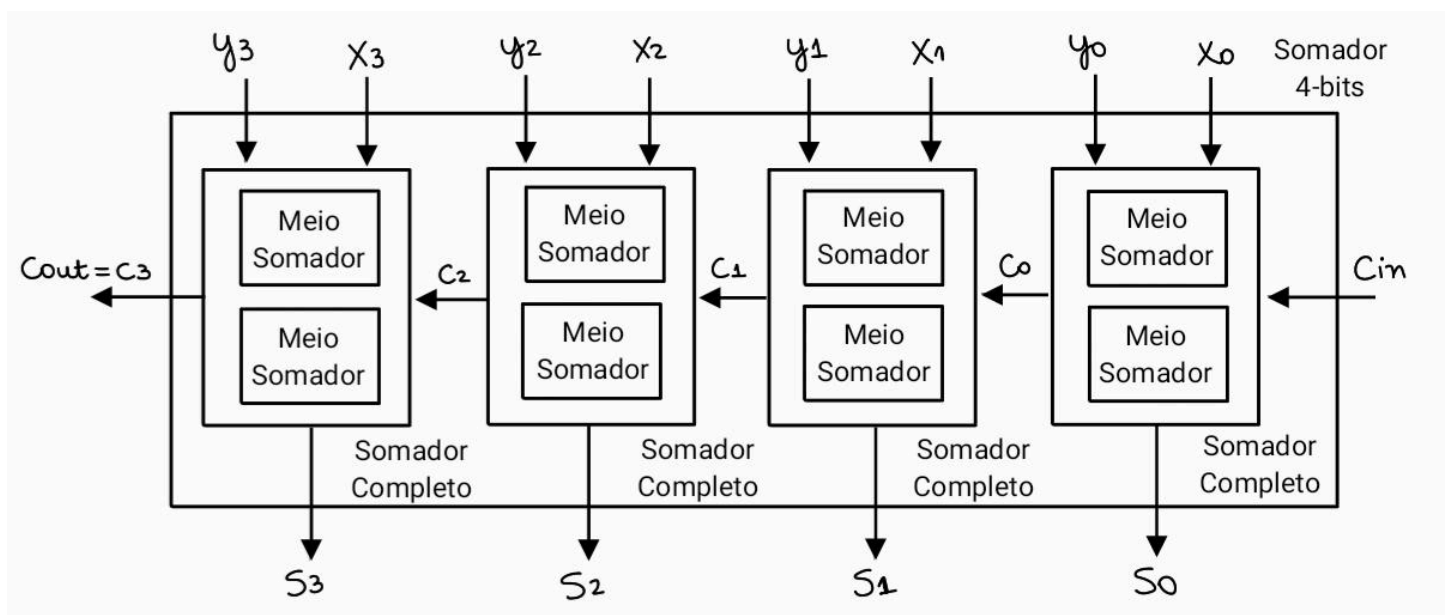


Figura 3.1: Esquemático do “Somador de 4 bits”.

Tabela Verdade do meio-somador:

y	x	g	p
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Esquemático do meio-somador:

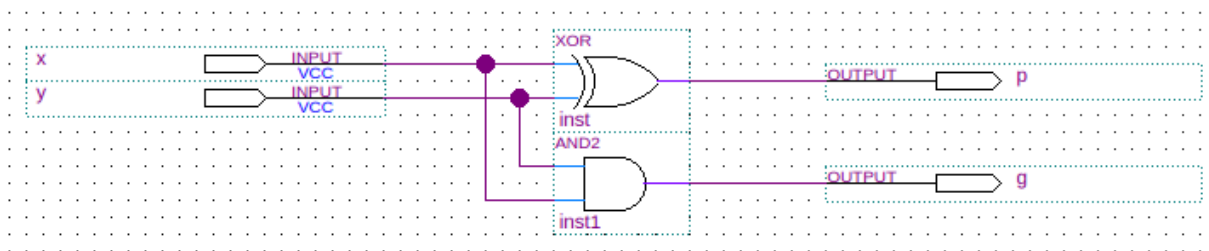


Figura 3.2: Diagrama esquemático do circuito meio-somador.

Tabela Verdade do somador completo:

cin	y	x	co	z
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Esquemático do somador completo:

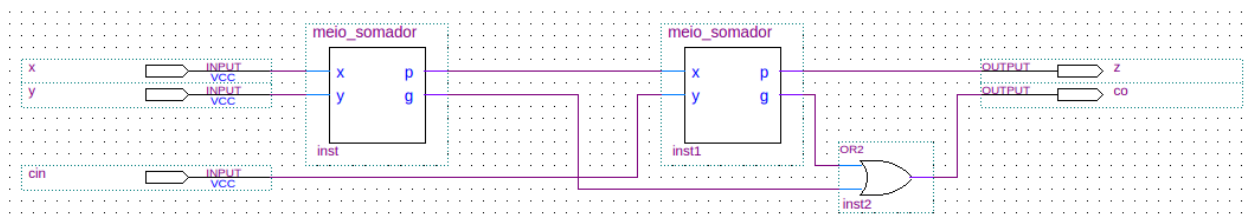


Figura 3.3: Diagrama esquemático do circuito somador completo.

Simulação funcional do somador completo:

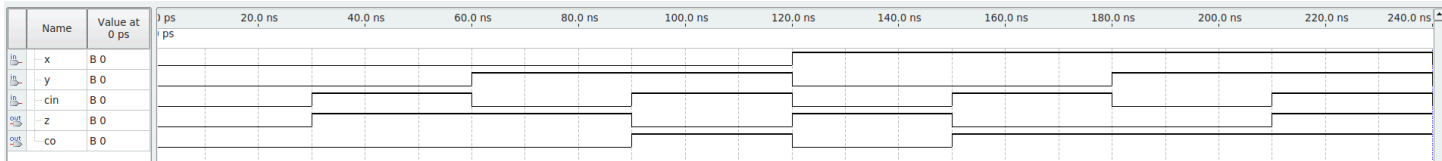


Figura 3.4: Resultado da simulação funcional para o somador completo.

Agora, podemos voltar para as flags:

Tabela verdade de C:

c_3	c_2	c_1	c_0	C
0	x	x	x	0
1	x	x	x	1

Tabela Verdade de V:

c_3	c_2	c_1	c_0	V
0	0	x	x	0
0	1	x	x	1
1	0	x	x	1
1	1	x	x	0

Tabela Verdade de N:

s_3	s_2	s_1	s_0	N
0	x	x	x	0
1	x	x	x	1

Tabela Verdade de Z:

s_3	s_2	s_1	s_0	Z
0	0	0	0	1
x	x	x	1	0
x	x	1	x	0
x	1	x	x	0
1	x	x	x	0

4.1.2. Minimizações

É fácil perceber que para as flags C e N, temos:

$$C = c_3$$

$$N = s_3$$

4.1.2.1. Soma de Produtos

Z

	$\bar{s}_1 \cdot \bar{s}_0$	$\bar{s}_1 \cdot s_0$	$s_1 \cdot s_0$	$s_1 \cdot \bar{s}_0$
$\bar{s}_3 \cdot \bar{s}_2$	1	0	0	0
$\bar{s}_3 \cdot s_2$	0	0	0	0
$s_3 \cdot s_2$	0	0	0	0
$s_3 \cdot \bar{s}_2$	0	0	0	0

$$Z = \bar{s}_3 \cdot \bar{s}_2 \cdot \bar{s}_1 \cdot \bar{s}_0 \text{ (4 NOTs, 1 AND)(5 Portas)}$$

Podemos ainda simplificar a expressão encontrada por meio do Teorema de Morgan:

$$Z = \overline{s_3 + s_2 + s_1 + s_0} \text{ (1 NOR)(1 Porta)}$$

V

	$\bar{c}_1 \cdot \bar{c}_0$	$\bar{c}_1 \cdot c_0$	$c_1 \cdot c_0$	$c_1 \cdot \bar{c}_0$
$\bar{c}_3 \cdot \bar{c}_2$	0	0	0	0
$\bar{c}_3 \cdot c_2$	1	1	1	1
$c_3 \cdot c_2$	0	0	0	0
$c_3 \cdot \bar{c}_2$	1	1	1	1

$$V = \bar{c}_3 \cdot c_2 + c_3 \cdot \bar{c}_2 \text{ (2 ANDs, 1 OR, 2 NOTs)(5 Portas)}$$

A expressão acima pode ainda ser simplificada utilizando um XOR:

$$V = \bar{c}_3 \cdot c_2 + c_3 \cdot \bar{c}_2 = c_3 \oplus c_2$$

Logo, temos que:

$$V = c_3 \oplus c_2 \text{ (1 XOR)(1 Porta)}$$

4.1.2.2. Produto de Somas

Z

	$s_1 + s_0$	$s_1 + \bar{s}_0$	$\bar{s}_1 + \bar{s}_0$	$\bar{s}_1 + s_0$
$s_3 + s_2$	1	0	0	0
$s_3 + \bar{s}_2$	0	0	0	0
$\bar{s}_3 + \bar{s}_2$	0	0	0	0
$\bar{s}_3 + s_2$	0	0	0	0

$$Z = \bar{s}_3 \cdot \bar{s}_2 \cdot \bar{s}_1 \cdot \bar{s}_0 \text{ (4 NOTs, 1 AND)(5 Portas)}$$

Podemos ainda simplificar a expressão encontrada por meio do Teorema de Morgan:

$$Z = \overline{s_3 + s_2 + s_1 + s_0} \text{ (1 NOR)(1 Porta)}$$

V

	$c_1 + c_0$	$c_1 + \bar{c}_0$	$\bar{c}_1 + \bar{c}_0$	$\bar{c}_1 + c_0$
$c_3 + c_2$	0	0	0	0
$c_3 + \bar{c}_2$	1	1	1	1
$\bar{c}_3 + \bar{c}_2$	0	0	0	0
$\bar{c}_3 + c_2$	1	1	1	1

$$V = (c_3 + c_2) \cdot (\bar{c}_3 + \bar{c}_2) \text{ (2 ORs, 1 AND, 2 NOTs)(5 Portas)}$$

4.1.2.3. Esquemático do Circuito

O circuito será implementado com as expressões encontradas para C e N, além da simplificação encontrada para Z com a porta NOR. Para V utilizaremos a função Produto de Somas, devido a facilidade de se usar a porta XOR.

Na Figura 3.5 é apresentado o diagrama esquemático do circuito projetado.

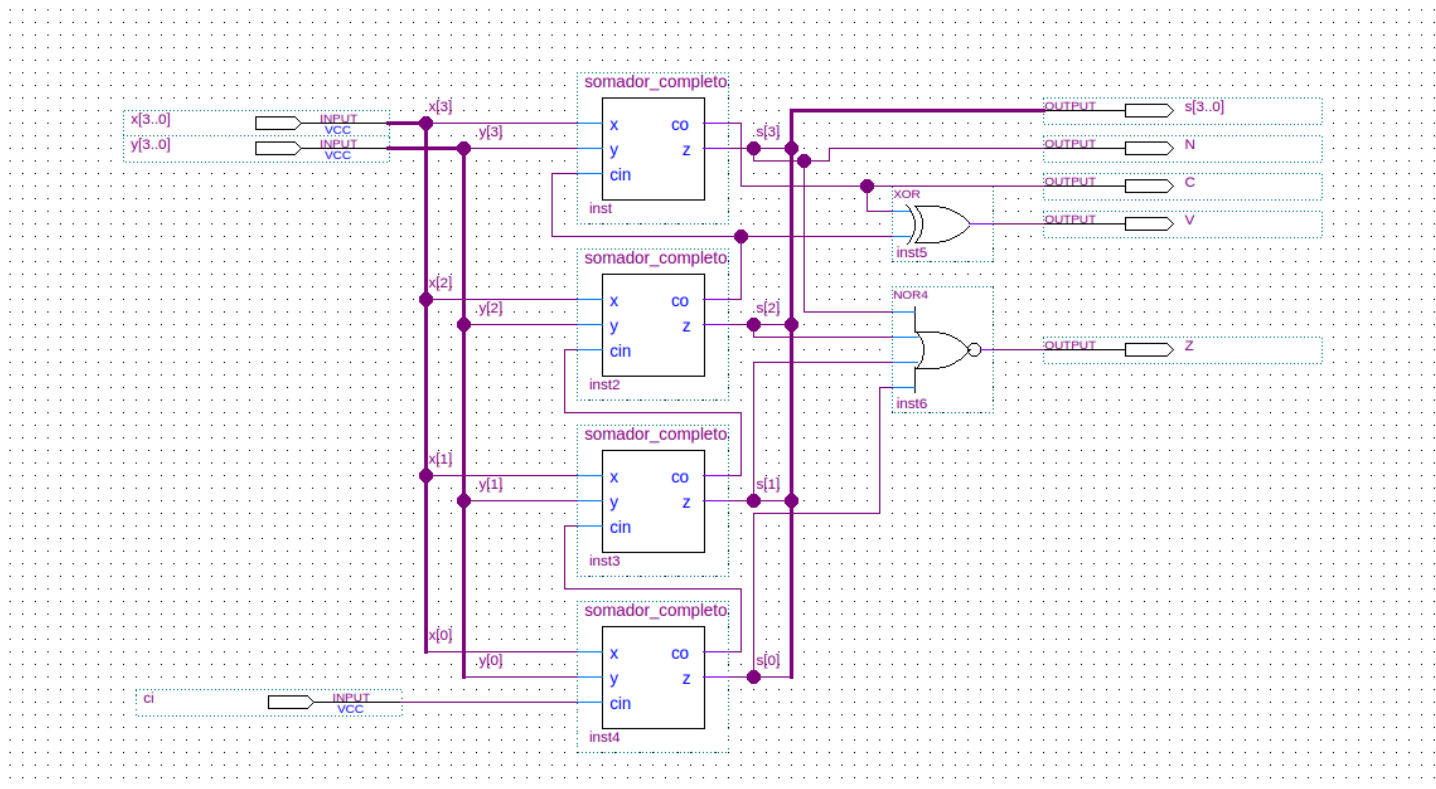


Figura 3.5: Diagrama esquemático para o módulo “Somador de 4 bits”.

4.2. Simulação

Para a simulação, foram realizadas diferentes combinações entre as entradas $x[3..0]$, $y[3..0]$ e ci , de modo a mostrar as ativações e diferentes combinações entre as “flags” do Somador de 4 bits.

4.2.1. Simulação Funcional

A Figura 3.6 apresenta as formas de ondas dos sinais de entrada e saída geradas pela simulação funcional.

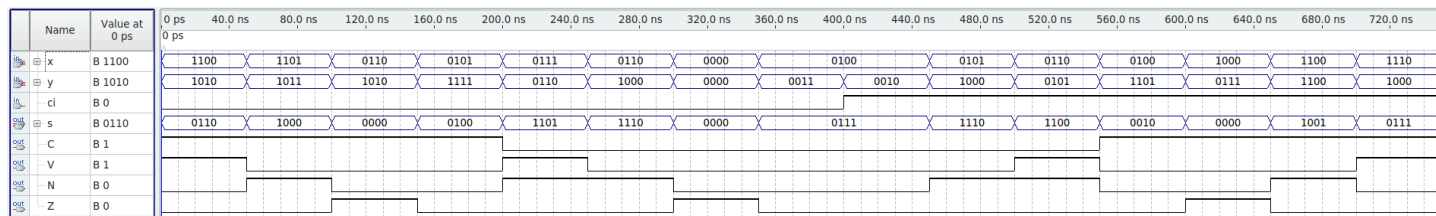


Figura 3.6: Resultado da simulação funcional para o módulo “Somador de 4 bits”.

4.2.2. Simulação Temporal

A Figura 3.7 apresenta as formas de ondas dos sinais de entrada e sinal de saída geradas pela simulação temporal, que contempla os atrasos das portas. Da mesma forma que o resultado da simulação funcional, a simulação temporizada também indica a operação do circuito consoante a especificação.

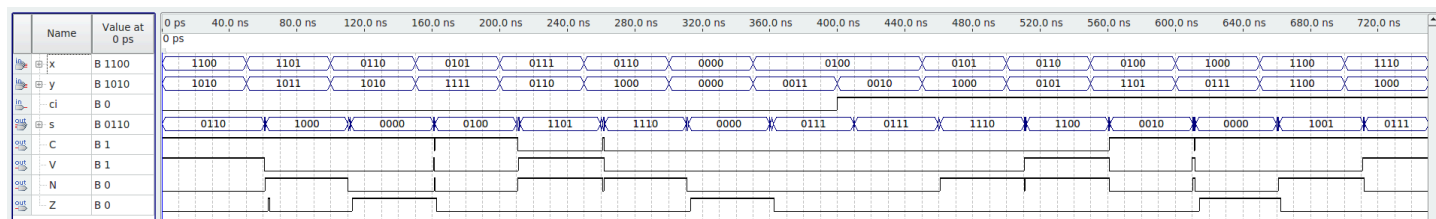


Figura 3.7: Resultado da simulação temporizada para o módulo “Somador de 4 bits”.

5. O módulo “Decodificador ULA”

O módulo “Decodificador ULA” é um circuito combinacional que tem por objetivo selecionar e controlar as operações aritméticas a serem realizadas. O decodificador recebe como entrada um carry (ci) de 1 bit e um código de 4 bits cuja as operações, para cada valor, já estão pré definidas. A partir daí o decodificador é capaz de controlar os outros módulos da ULA a fim de realizar a operação definida pelo código de entrada.

5.1. Projeto do Circuito

5.1.1. Especificação Binária

Entrada:

$f_n \in \{0, 1\}$, com $n \in \{0, 1, 2, 3\}$ - as entradas f são as entradas de seleção, são os códigos de 4 bits que especificam a operação aritmética a ser realizada pela ULA.

$ci \in \{0, 1\}$ - ci é o carry de entrada.

Saída:

$cmpx, zry, cmpy, cci \in \{0, 1\}$

As saídas possuem as seguintes especificações:

cmpx: ativa o módulo “Complementa” de x, que complementa as entradas x, bit-a-bit;

zry: ativa o módulo “Zera” que zera as entradas de y;

cmpy: ativa o módulo “Complementa” de y, que complementa as entradas y, bit-a-bit;

cci: controla o sinal do carry de entrada no Somador de 4 bits.

Função:

Código ($f_3 f_2 f_1 f_0$)	Nome	Operação	Descrição	Saídas
000x	ADD	$s = x + y$	adição de x com y	cmpx = 0, zry = 0, cmpy = 0, cci = 0
001x	ADDC	$s = x + y + ci$	adição de x com y e carry	cmpx = 0, zry = 0, cmpy = 0, cci = ci
010x	SUB	$s = x - y$	subtração y de x	cmpx = 0, zry = 0, cmpy = 1, cci = 1
0110	INC	$s = x + 1$	incremento unitário de x	cmpx = 0, zry = 1, cmpy = 0, cci = 1
0111	DEC	$s = x - 1$	decremento unitário de x	cmpx = 0, zry = 1, cmpy = 1, cci = 0
1000	NEG	$s = -x$	negação aritmética de x em complemento de 2	cmpx = 1, zry = 1, cmpy = 0, cci = 1
1001	CMPL	$s = \sim x$	complemento bit-a-bit de x	cmpx = 1, zry = 1, cmpy = 0, cci = 0

Tabela 5.1: Especificação dos códigos (entradas f).

Tabela Verdade:

f_3	f_2	f_1	f_0	cmpx	zry	cmpy	cci
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	ci
0	0	1	1	0	0	0	ci
0	1	0	0	0	0	1	1
0	1	0	1	0	0	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	1	0
1	0	0	0	1	1	0	1
1	0	0	1	1	1	0	0
1	0	1	0	x	x	x	x
1	0	1	1	x	x	x	x
1	1	0	0	x	x	x	x
1	1	0	1	x	x	x	x
1	1	1	0	x	x	x	x
1	1	1	1	x	x	x	x

5.1.2. Minimizações

5.1.2.1. Soma de Produtos

cmpx

	$\overline{f_1}.\overline{f_0}$	$\overline{f_1}.f_0$	$f_1.\overline{f_0}$	$f_1.f_0$
$\overline{f_3}.\overline{f_2}$	0	0	0	0
$\overline{f_3}.f_2$	0	0	0	0
$f_3.\overline{f_2}$	x	x	x	x
$f_3.f_2$	1	1	x	x

$$cmpx = f_3$$

zry

	$\overline{f_1} \cdot \overline{f_0}$	$\overline{f_1} \cdot f_0$	$f_1 \cdot f_0$	$f_1 \cdot \overline{f_0}$
$\overline{f_3} \cdot \overline{f_2}$	0	0	0	0
$\overline{f_3} \cdot f_2$	0	0	1	1
$f_3 \cdot f_2$	x	x	x	x
$f_3 \cdot \overline{f_2}$	1	1	x	x

$$zry = f_3 + f_2 \cdot f_1 \text{ (1 AND, 1 OR)(2 Portas)}$$

cmpr

	$\overline{f_1} \cdot \overline{f_0}$	$\overline{f_1} \cdot f_0$	$f_1 \cdot f_0$	$f_1 \cdot \overline{f_0}$
$\overline{f_3} \cdot \overline{f_2}$	0	0	0	0
$\overline{f_3} \cdot f_2$	1	1	1	0
$f_3 \cdot f_2$	x	x	x	x
$f_3 \cdot \overline{f_2}$	0	0	x	x

$$cmpr = f_2 \cdot \overline{f_1} + f_2 \cdot f_0 \text{ (2 ANDs, 1 OR, 1 NOT)(4 Portas)}$$

cci

	$\overline{f_1} \cdot \overline{f_0}$	$\overline{f_1} \cdot f_0$	$f_1 \cdot f_0$	$f_1 \cdot \overline{f_0}$
$\overline{f_3} \cdot \overline{f_2}$	0	0	ci	ci
$\overline{f_3} \cdot f_2$	1	1	0	1
$f_3 \cdot f_2$	x	x	x	x
$f_3 \cdot \overline{f_2}$	1	0	x	x

$$cci = f_2 \cdot \overline{f_1} + f_2 \cdot \overline{f_0} + f_3 \cdot \overline{f_0} + ci \cdot \overline{f_2} \cdot f_1 \text{ (4 ANDs, 1 OR, 3 NOTs)(8 Portas)}$$

5.1.2.2. Produto de Somas

cmpx

	$f_1 + f_0$	$f_1 + \bar{f}_0$	$\bar{f}_1 + \bar{f}_0$	$\bar{f}_1 + f_0$
$f_3 + f_2$	0	0	0	0
$f_3 + \bar{f}_2$	0	0	0	0
$\bar{f}_3 + \bar{f}_2$	x	x	x	x
$\bar{f}_3 + f_2$	1	x	x	x

$$cmpx = f_3$$

zry

	$f_1 + f_0$	$f_1 + \bar{f}_0$	$\bar{f}_1 + \bar{f}_0$	$\bar{f}_1 + f_0$
$f_3 + f_2$	0	0	0	0
$f_3 + \bar{f}_2$	0	0	1	1
$\bar{f}_3 + \bar{f}_2$	x	x	x	x
$\bar{f}_3 + f_2$	1	1	x	x

$$zry = (f_3 + f_2) \cdot (f_3 + f_1) \text{ (2 ORs, 1 AND)(3 Portas)}$$

cmpy

	$f_1 + f_0$	$f_1 + \bar{f}_0$	$\bar{f}_1 + \bar{f}_0$	$\bar{f}_1 + f_0$
$f_3 + f_2$	0	0	0	0
$f_3 + \bar{f}_2$	1	1	1	0
$\bar{f}_3 + \bar{f}_2$	x	x	x	x
$\bar{f}_3 + f_2$	0	0	x	x

$$cmpy = f_2 \cdot (\bar{f}_1 + f_0) \text{ (1 OR, 1 AND, 1 NOT)(3 Portas)}$$

	$f_1 + f_0$	$f_1 + \overline{f_0}$	$\overline{f_1} + \overline{f_0}$	$\overline{f_1} + f_0$
$f_3 + f_2$	0	0	ci	ci
$f_3 + \overline{f_2}$	1	1	0	1
$\overline{f_3} + \overline{f_2}$	x	x	x	x
$\overline{f_3} + f_2$	1	0	x	x

$$cci = (\overline{f_0} + \overline{f_3}) \cdot (\overline{f_2} + \overline{f_1} + \overline{f_0}) \cdot (f_3 + f_2 + f_1) \cdot (ci + f_2 + \overline{f_1}) \quad (4 \text{ ORs, } 1 \text{ AND, } 4 \text{ NOTs})(9 \text{ Portas})$$

5.1.2.3. Esquemático do Circuito

Na implementação do circuito, as funções foram selecionadas tendo em vista a utilização do menor número de portas lógicas possível. Para cmpx qualquer uma das funções pode ser usada, já que ambas são iguais; soma de produtos para zry; produto de somas para cmpy; soma de produtos para cci.

A Figura 5.1 apresenta o diagrama esquemático do circuito projetado.

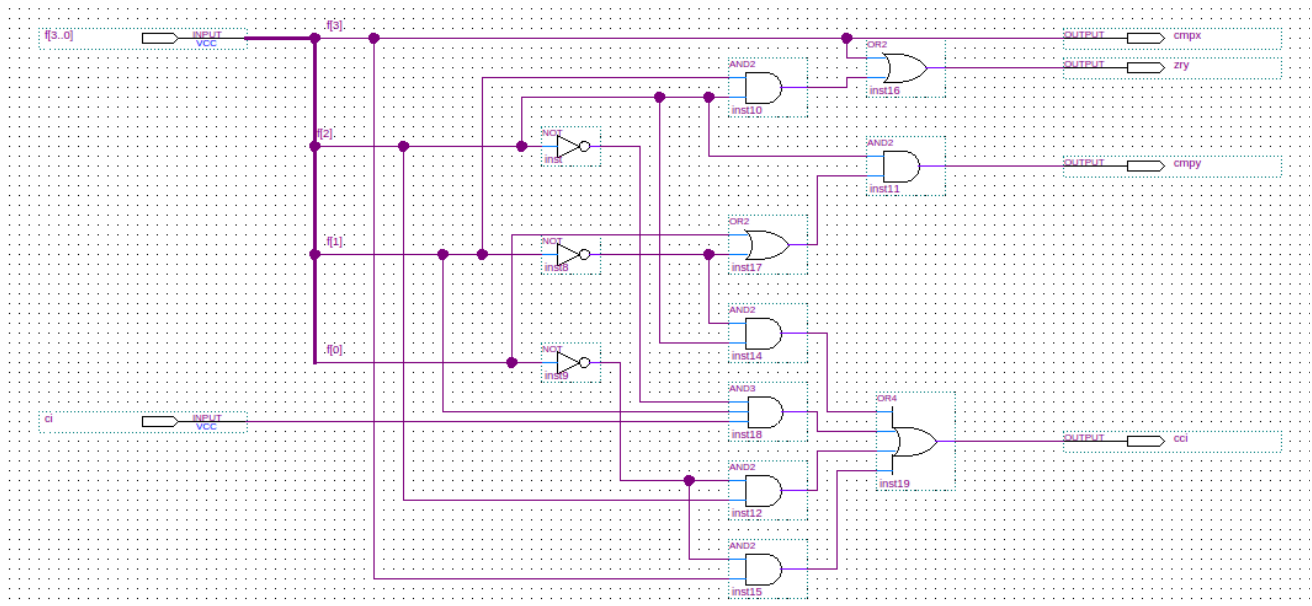


Figura 5.1: Diagrama esquemático do circuito projetado para o módulo “Decodificador ULA”.

5.2. Simulação

Para a simulação, as entradas foram configuradas de modo a representar todas as combinações possíveis entre ci e as entradas f que estão definidas na Tabela 5.1. Foram descartados os valores de 1010 a 1111, já que esses não são especificados no funcionamento da ULA.

5.2.1. Simulação Funcional

A Figura 5.2 apresenta as formas de ondas dos sinais de entrada e saída geradas pela simulação funcional. Para cada valor de f entre 0000 e 1001, são ativadas ou desativadas as respectivas saídas que implementam as operações aritméticas da ULA, assim como especificado na Tabela 5.1.

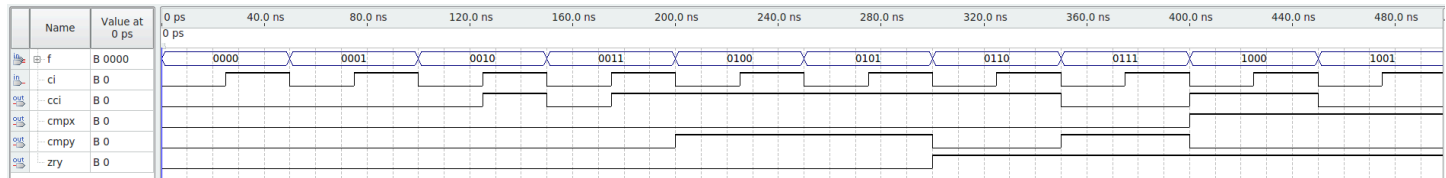


Figura 5.2: Resultado da simulação funcional para o módulo “Decodificador ULA”.

5.2.2. Simulação Temporal

A Figura 5.3 apresenta as formas de ondas dos sinais de entrada e sinal de saída geradas pela simulação temporal, que contempla os atrasos das portas. Da mesma forma que o resultado da simulação funcional, a simulação temporizada também indica a operação do circuito consoante a especificação.

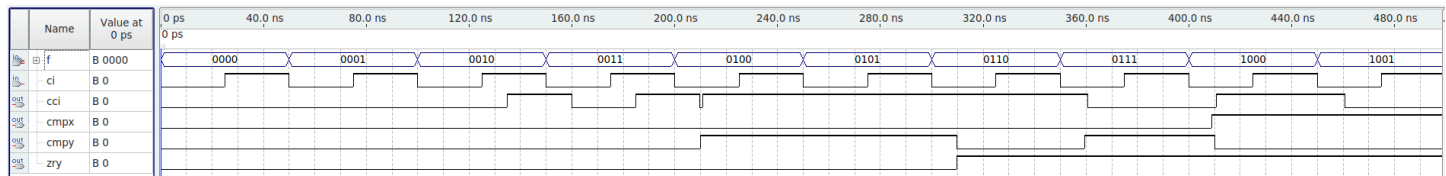


Figura 5.3: Resultado da simulação temporizada para o módulo “Decodificador ULA”.

6. A Unidade Lógica e Aritmética (ULA)

Agora, tendo todos os módulos prontos e definidos, basta juntá-los para obtermos nossa ULA.

6.1. Projeto do Circuito

6.1.1. Especificação Binária

Entrada:

$x_n, y_n, f_n \in \{0, 1\}, \text{ com } n \in \{0, 1, 2, 3\}$
 $ci \in \{0, 1\}$

Saída:

$s_n \in \{0, 1\}, \text{ com } n \in \{0, 1, 2, 3\}$
 $C, V, N, Z \in \{0, 1\}$

Função:

A ULA tem como função realizar as operações especificadas no módulo “Decodificador ULA” (vide Tabela 5.1), e apresentar o resultado da operação aritmética e as 4 “flags” especificadas no módulo “Somador de 4 bits”.

Unindo todos os módulos da ULA, temos:

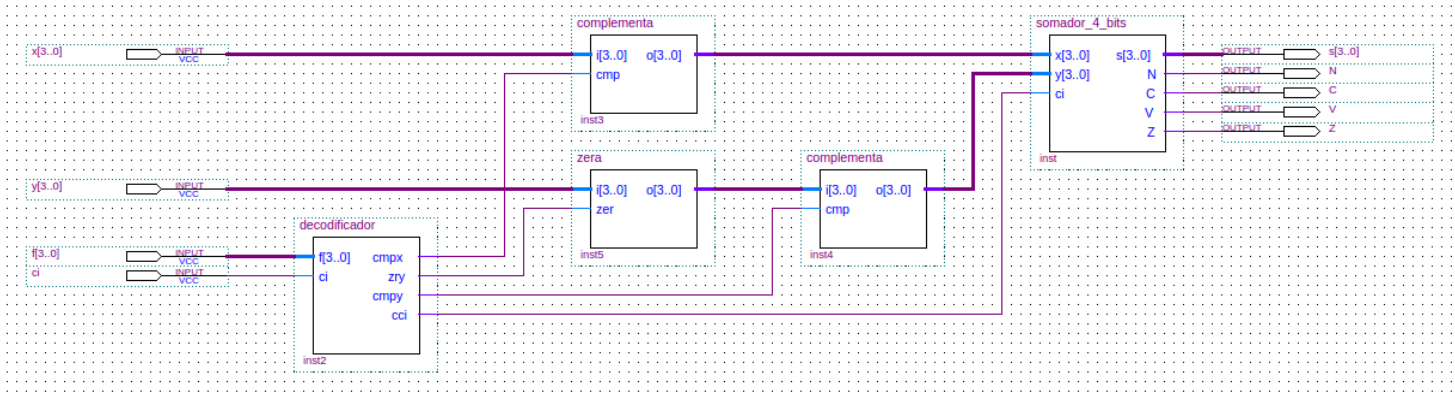


Figura 6.1: Diagrama esquemático da ULA, apresentando todos os módulos e suas conexões.

6.1.2. Esquemático do Circuito

O circuito apresentado na Figura 6.1 pode ainda ser encapsulado para termos apenas um bloco, denominado “ula”, e suas respectivas entradas e saídas. O circuito final resultante, teve suas entradas e saídas atribuídas a pinos da placa de desenvolvimento utilizada. Atribuições:

Entradas	Saídas
$x_3, x_2, x_1, x_0 \rightarrow SW[3], SW[2], SW[1], SW[0]$	$s_3, s_2, s_1, s_0 \rightarrow LEDR[3], LEDR[2], LEDR[1], LEDR[0]$
$y_3, y_2, y_1, y_0 \rightarrow SW[7], SW[6], SW[5], SW[4]$	$C \rightarrow LEDR[4]$
$f_3, f_2, f_1, f_0 \rightarrow KEY[3], KEY[2], KEY[1] \text{ e } KEY[0]$	$V \rightarrow LEDR[5]$
$ci \rightarrow SW[8]$	$N \rightarrow LEDR[6]$
	$Z \rightarrow LEDR[7]$

A Figura 6.2 apresenta o diagrama esquemático do encapsulamento e das atribuições feitas aos pinos.

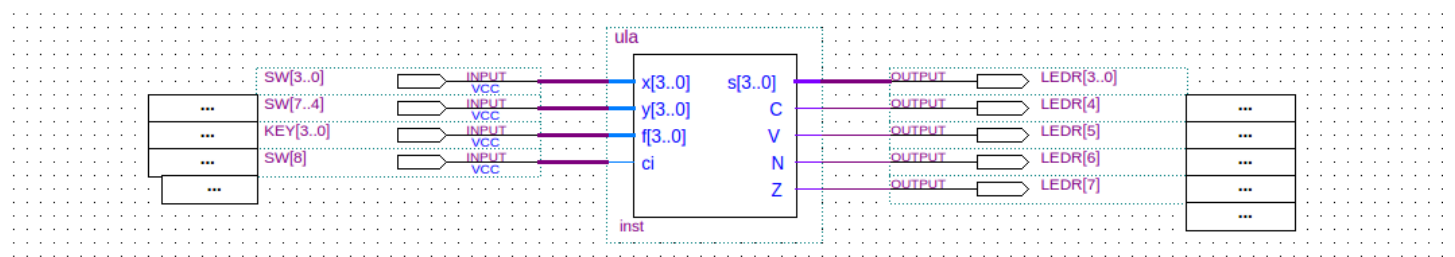


Figura 6.2: Diagrama esquemático do circuito final da ULA, com as entradas e saídas associadas às portas da FPGA Cyclone II, da Altera.

6.2. Simulação

Para a simulação, as entradas foram configuradas de modo a representar todas as configurações possíveis para as entradas f, e configurações aleatórias para as entradas x e y de modo que seja possível visualizar as diferentes possibilidades existentes para as saídas, realizando 3 operações distintas para cada função (especificadas pelas entradas f) da ULA.

6.2.1. Simulação Funcional

A Figura 6.3 apresenta as formas de onda dos sinais de entrada e saída geradas pela simulação funcional. As entradas x[3..0] e y[3..0] representam os números binários de 4 bits com os quais serão realizadas as operações aritméticas, a entrada ci representa o carry de entrada que também poderá participar das operações, as entradas f[3..0] representam os códigos/instruções que indicam a operação aritmética específica a ser realizada. As saídas s[3..0] representam o resultado da operação aritmética, e as saídas C, V, N e Z são as “flags”, já especificadas anteriormente, que são ativadas ou não, dependendo do resultado da operação.

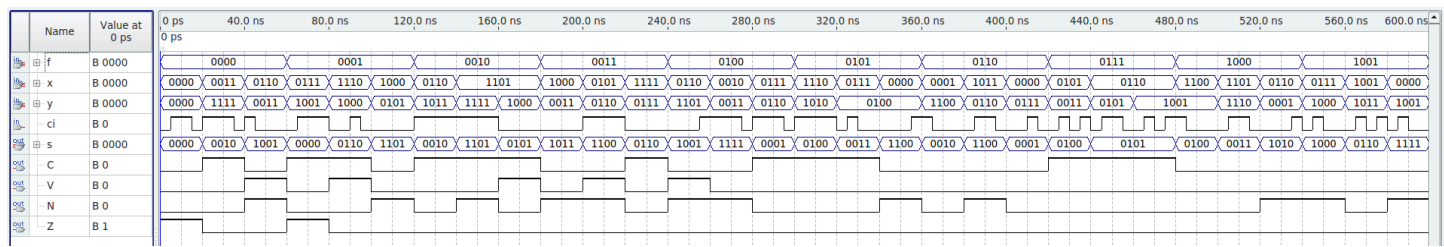


Figura 6.3: Resultado da simulação funcional para a Unidade Lógica e Aritmética.

6.2.2. Simulação Temporal

A Figura 6.4 apresenta as formas de ondas dos sinais de entrada e sinal de saída geradas pela simulação temporal, que contempla os atrasos das portas. Da mesma forma que o resultado da simulação funcional, a simulação temporizada também indica a operação do circuito consoante a especificação.

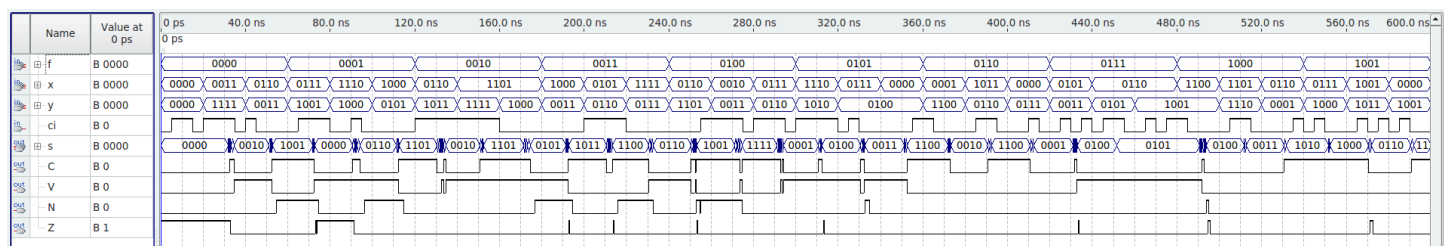


Figura 6.4: Resultado da simulação temporizada para a Unidade Lógica e Aritmética.